

3

Building the Context-Aware Multi-Agent System

In the last chapter, we created a **multi-agent system (MAS)** using MCP. It worked, but it came with one big limitation: its knowledge was simulated. That was fine for learning, but it also meant the agents were stuck inside a sandbox that we predefined. They couldn't reach out for real information or adapt their behavior in the moment. In this chapter, we're going to break them out of that box. Our agents will finally connect to external data through a live information pipeline built on a vector store.

This is where things get exciting. We're about to introduce a more powerful architecture: the **dual RAG MAS**. It uses **Retrieval-Augmented Generation (RAG)** in two ways. The first is what you'd expect—retrieving facts from a knowledge base. But the second is new: retrieving **procedural instructions**, instructions to implement **procedural (or dynamic) RAG**. This addition changes the game. Now, the system can learn not only *what* to say but also *how* to say it, switching style and structure on demand.

We'll build this step by step across two notebooks. The first focuses on data ingestion, where we'll create a knowledge base for *facts* and a context library for *semantic blueprints*. The second brings everything to life in runtime execution, showing how agents retrieve and combine both kinds of context in real time. Along the way, we will introduce you to a new specialist agent, the **Context Librarian**, and we'll see how the Orchestrator keeps the whole team working in sync.

This chapter covers the following topics:

- Architecting a dual RAG MAS
- Separating factual knowledge from procedural instructions
- Building a knowledge base for facts
- Creating a context library for semantic blueprints
- Using vector store namespaces to manage different data types
- Implementing new specialist agents, such as the Context Librarian
- Orchestrating the full system to generate context-aware content

We will begin by exploring the RAG MAS architecture.

Architecting a dual RAG MAS

To build our context-aware system, we need a clear picture of its architecture. At a high level, the entire process is divided into two distinct phases that we will design and develop. Phase 1 is **data preparation**, which prepares the data. Phase 2 is **runtime execution**, where our MAS uses the prepared data to respond to a user goal. This chapter develops both phases in two corresponding notebooks.

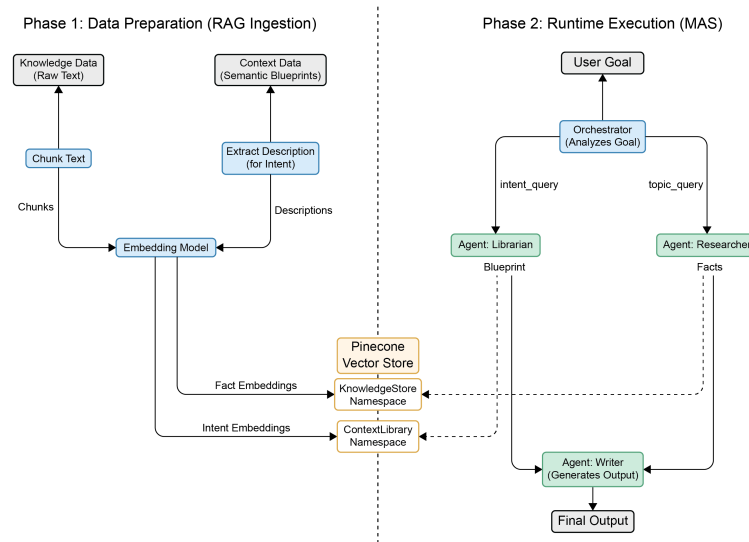


Figure 3.1: The dual RAG MAS architecture

The preceding figure illustrates the full data and execution flow of the dual RAG MAS architecture, with the arrows showing how the components connect at a high level. We'll walk through each part of the diagram before moving on to the actual programs.

Phase 1: Data preparation

In Phase 1, we focus on preparing the data, shown in the left section of Figure 3.1. The system begins with two distinct types of input sources:

- **Knowledge data:** Factual information that the system should know
- **Context data:** Structured instructions, or semantic blueprints, introduced in [Chapter 1](#)

Both data types are processed by the **embedding model**. For knowledge data, we split the text into chunks and embed each chunk. For context data or semantic blueprints, we embed only the description of the blueprint's intent. Searches are then based on intent. The complete blueprint content is stored separately in a JSON object, linked to its description.

All embeddings are stored in a Pinecone vector database, which will hold everything the system needs. We use a single Pinecone index, divided into two strictly separated namespaces:

- **KnowledgeStore**: This namespace stores the vectors from the factual data
- **ContextLibrary**: This namespace stores the vectors from the procedural blueprints

With both namespaces in place, the system is ready to move into Phase 2.

Phase 2: Runtime execution analysis

In Phase 2, the MAS executes a user request, as shown on the right side of Figure 3.1.

The process begins with a **user goal**, a high-level instruction submitted by the user—for example, `Write a suspenseful story about Apollo 11.`

The **Orchestrator** receives this goal. Acting as the system's central coordinator, it analyzes the request and identifies two components:

- `intent_query` : This is the intent of the Librarian query and relates to the desired style or structure (in our example, `suspenseful story`)
- `topic_query` : This is the topic of the Researcher query and relates to the subject matter (in our example, `Apollo 11`)

The Orchestrator then delegates each query to a specialist agent:

- The **Librarian agent**, responsible for retrieving instruction scripts, receives the `intent_query` through the engine's MCP messaging layer. It queries the `ContextLibrary` namespace, performs a semantic search to find the blueprint description that best matches the requested intent, and retrieves the corresponding semantic blueprint.
- The **Researcher agent**, responsible for retrieving factual information, receives the `topic_query`. It queries the `KnowledgeStore` namespace, retrieves the relevant factual chunks, and synthesizes them into concise findings.

Once both responses are ready, the Orchestrator can now facilitate the delivery of the retrieved instructions script and the synthesized facts to the **Writer** agent. The arrows in *Figure 3.1* show the **blueprint** and **facts** moving from the Librarian and Researcher toward the Writer.

The Writer acts as the system's generation engine. It uses the facts as the content and the blueprint as the strict instructions for structuring and styling that content. The Writer then generates the final output, adhering to both the factual requirements and the procedural constraints.

This architecture allows for high flexibility and scaling. The knowledge base can be updated without altering the instruction scripts, and new blueprints can be added without touching the facts. At runtime, the agents adapt their behavior dynamically based on the context they retrieve.

With the architecture explained, we can now move to implementation. The next step is to build the RAG ingestion pipeline.

RAG pipeline data ingestion (context and knowledge)

In **Chapter 2**, we built an MAS using simulated data. The agents knew how to search, but they were limited to a simple Python dictionary. In this chapter, we take the next step: replacing that simulation with a real RAG pipeline.

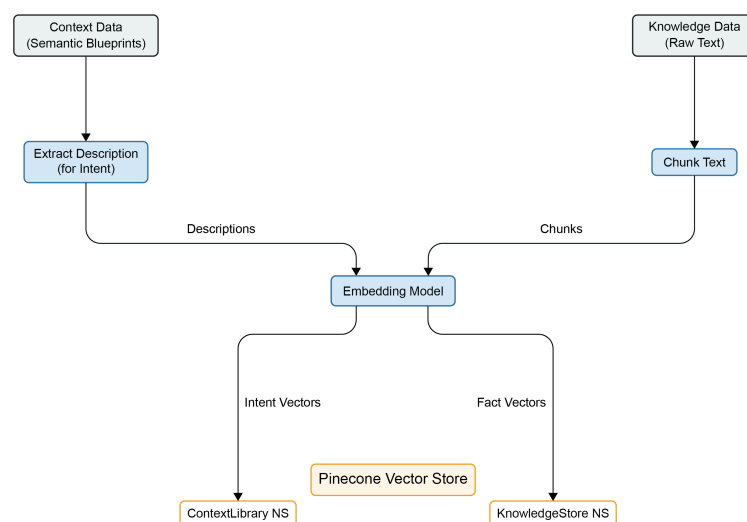


Figure 3.2: Flowchart of the RAG ingestion pipeline

Figure 3.2 shows the flow of this pipeline. Before the agents can search effectively, we must first prepare and build the databases they will use. Our approach introduces two complementary forms of RAG:

- **Knowledge base (factual RAG):** Stores facts. This is the standard usage of RAG, familiar to most engineers.
- **Context library (procedural RAG):** Stores instructions or **semantic blueprints** introduced in [Chapter 1](#). Agents query this library to determine *how* to structure their responses and *what style* to use.

To implement this, open the `RAG_Pipeline.ipynb` notebook in the GitHub repository for this chapter. In it, we will build both databases, extract and convert the data into embeddings, and finally, upload the embeddings into a **Pinecone vector store**, keeping the two databases separate through namespaces.

Installation and setup

Let's start by preparing the environment. For this project, I'll be using **Pinecone** as the vector database, but you can substitute another platform if that better suits your needs. To keep things practical, we'll also bring in **tqdm** (to monitor upload progress) and **Tenacity** (to make API calls more reliable when networks inevitably hiccup).

Now, one lesson I've learned the hard way: Python's library ecosystem changes quickly. If we don't freeze versions, we can end up wrestling with dependency errors instead of building our system. To save you from that pain, I'm freezing specific versions here:

```
!pip install tqdm==4.67.1 --upgrade
!pip install openai==1.104.2
!pip install pinecone==7.0.0 tqdm==4.67.1 tenacity==8.3.0
```

Next, we need to import the necessary modules into our notebook:

```
# Imports for this notebook
import json
import time
from tqdm.auto import tqdm
import tiktoken
from pinecone import Pinecone, ServerlessSpec
from tenacity import retry, stop_after_attempt, wait_random_exponential
# general imports required in the notebooks of this book
import re
import textwrap
from IPython.display import display, Markdown
import copy
```

To use **OpenAI** and **Pinecone**, you'll need API keys. If you haven't signed up yet, both platforms make it straightforward, and Pinecone even offers a free plan with a limited number of indexes (perfect for experimenting). In this notebook, I'm using **Google Colab's Secrets Manager** to provide the keys, but if you're running locally, the code will fall back to environment variables. Feel free to adapt this to your own setup.

For embeddings, I'm using OpenAI's **text-embedding-3-small** model. It produces 1,536-dimensional vectors, which is plenty for our purposes. You could choose another embedding model, but consistency is key: whatever you use for creating embeddings must also be used for retrieval. We'll lock these details in variables right after setting up the API keys:

```
import os
from openai import OpenAI
from google.colab import userdata

# Load the API key from Colab secrets, set the env var, then init the client
...
# Configuration
EMBEDDING_MODEL = "text-embedding-3-small"
EMBEDDING_DIM = 1536 # Dimension for text-embedding-3-small
GENERATION_MODEL = "gpt-5"
```

Note on model choice: We use GPT-5 for continuity across this and upcoming chapters, but the engine is model-agnostic; you can switch to any compliant LLM (e.g., Mistral or Claude) by updating the `GENERATION_MODEL` setting. The rest of the architecture (prompts, registry, and planner/executor) stays the same.

If everything is set up correctly, the notebook will reassure you with a message such as the following:

```
OpenAI API key loaded and environment variable set successfully.
```

With that done, we're ready to move on and initialize the Pinecone index.

Initializing the Pinecone index

Now that our environment is ready, let's connect to OpenAI and Pinecone using the API keys we just set up. The first step is to define a name for our Pinecone index. For this chapter, we'll call it `genai-mas-mcp-ch3`.

We'll then retrieve the Pinecone API key. In this example, I'm using Google Colab's Secrets Manager:

```
try:
    # Standard way to access secrets securely in Google Colab
    from google.colab import userdata
    PINECONE_API_KEY = userdata.get('PINECONE_API_KEY')
    ...
```

Once we have the API key, we need to define the two namespaces for this project. These will keep our two data types strictly separated inside Pinecone:

- `NAMESPACE_KNOWLEDGE` will store the facts
- `NAMESPACE_CONTEXT` will store the blueprints

We are using Pinecone's serverless architecture, which is recommended for Pinecone's free plan. Make sure to consult Pinecone's plans regularly to see whether the conditions change. For this demo, I'll use AWS in the `us-east-1` Region:

```
# --- Initialize Clients ---
client = OpenAI(api_key=OPENAI_API_KEY)
pc = Pinecone(api_key=PINECONE_API_KEY)

# --- Define Index and Namespaces ---
INDEX_NAME = 'genai-mas-mcp-ch3'
NAMESPACE_KNOWLEDGE = "KnowledgeStore"
NAMESPACE_CONTEXT = "ContextLibrary"

# Define Serverless Specification
spec = ServerlessSpec(cloud='aws', region='us-east-1')
```

Before we can use the index, we need to check whether it already exists. If it doesn't, the code creates a new one with the embedding dimension we defined earlier (1536) and sets the metric to **cosine similarity**, which is a common choice for comparing text embeddings:

```
# Check if index exists
if INDEX_NAME not in pc.list_indexes().names():
    print(f"Index '{INDEX_NAME}' not found. Creating new serverless index...")
    pc.create_index(
        name=INDEX_NAME,
        dimension=EMBEDDING_DIM,
        metric='cosine',
        spec=spec
    )
    # Wait for index to be ready
    while not pc.describe_index(INDEX_NAME).status['ready']:
        print("Waiting for index to be ready...")
        time.sleep(1)
    print("Index created successfully. It is new and empty.")
```

If the index already exists, we can choose whether to clear out existing data. For this demo, we'll start fresh each time by deleting the contents of both namespaces:

```
else:
    print(f"Index '{INDEX_NAME}' already exists. Clearing namespaces for a fresh start...")
    index = pc.Index(INDEX_NAME)
    namespaces_to_clear = [NAMESPACE_KNOWLEDGE, NAMESPACE_CONTEXT]
```

To clear the index, we need to go through each namespace and clear the data:

```
for namespace in namespaces_to_clear:
    # Check if namespace exists and has vectors before deleting
    stats = index.describe_index_stats()
    if namespace in stats.namespaces
    AND stats.namespaces[namespace].vector_count > 0:
        print(f"Clearing namespace '{namespace}'...")
        index.delete(delete_all=True, namespace=namespace)
```

There's a tricky part here. The `clearing` function can be asynchronous, and the program might continue too quickly. This means that a slow deletion can finish *after* our upload starts, and those new vectors could be swept away. It's not a certainty, but it's surely a risk we don't want to take.

So, we force the system to wait and only proceed once the vector count is truly zero:

```
# **CRITICAL FUNCTION: Wait for deletion to complete**
while True:
    stats = index.describe_index_stats()
    if namespace not in stats.namespaces or
    stats.namespaces[namespace].vector_count == 0:
        print(f"Namespace '{namespace}' cleared successfully.")
        break
    print(f"Waiting for namespace '{namespace}' to clear...")
    time.sleep(5) # Poll every 5 seconds
else:
    print(f"Namespace '{namespace}' is already empty or does not exist. Skipping.")
```

Finally, we are ready to connect to the index for subsequent operation:

```
# Connect to the index for subsequent operations
index = pc.Index(INDEX_NAME)
```

If everything runs correctly, you'll see output that indicates the index was already present and namespaces are cleared:

```
Index 'genai-mas-mcp-ch3' already exists. Clearing namespaces for a fresh start...
Namespace 'KnowledgeStore' is already empty or does not exist. Skipping.
Namespace 'ContextLibrary' is already empty or does not exist. Skipping.
```

In production, you may not want to clear namespaces on every run—sometimes you'll want the system to keep its stored knowledge. For our learning journey, however, it's cleaner to start fresh each time. With the index ready, we can move on to preparing the data.

Data preparation: the context library (procedural RAG)

In this section, we'll define the data for our context library. This is the heart of our procedural RAG implementation because, here, we're introducing procedural instructions—not just static knowledge. The context library builds on the semantic blueprint concept from [Chapter 1](#), but now we'll implement it with a precise structure.

Each entry in our context library will have three parts:

- An `id` that is a unique identifier in a Pinecone index.
- A `description` that will explain the purpose and style of the blueprint. We will only embed the description as a vector to find the blueprint we need for a task. Think of it as a library card for a book, not the book itself.
- The `blueprint` contains the *actual* instructions for the LLM in JSON format. The blueprint will *not* be embedded. Once the description is accessed, we will retrieve the blueprint like we would a book once we know where it is.

Here's how this plays out at runtime: when the Orchestrator needs a specific style, the Librarian agent searches for the description that best matches the request. Once found, it pulls the complete semantic blueprint in JSON form. This approach makes the system massively scalable—you can add as many LLM tasks as needed to your vector store without rewriting agents.

Let's now define three example blueprints here:

- One for suspenseful narratives (creative writing)
- One for technical explanations (factual and precise)
- One for casual summaries (a common LLM request)

Defining blueprints is a key context engineering skill. LLMs are not telepathic—they need carefully crafted instructions. Even if you rely on a copilot or API to generate drafts, you must still verify the quality of these descriptions. Consider our first example—the suspense narrative example. This blueprint is designed for storytelling. The goal is to build tension and keep the reader on edge:

```
context_blueprints = [
    {
        "id": "blueprint_suspense_narrative",
        "description": "A precise Semantic Blueprint designed to generate suspenseful and",
        "blueprint": json.dumps({
            "scene_goal": "Increase tension and create suspense.",
            "style_guide": "Use short, sharp sentences. Focus on sensory details (sounds",
            "participants": [
                { "role": "Agent", "description": "The protagonist experiencing the events",
                { "role": "Source_of_Threat", "description": "The underlying danger or mys
            ],
            "instruction": "Rewrite the provided facts into a narrative adhering strictly
        })
    },
```

Notice how specific the wording has to be. Writing a suspenseful children's story is very different from writing a technical description, and the blueprint needs to reflect that.

Here's the technical explanation for the blueprint. This blueprint is the opposite of the narrative one. Instead of atmosphere and emotion, it emphasizes clarity, structure, and objectivity:

```
{
    "id": "blueprint_technical_explanation",
    "description": "A Semantic Blueprint designed for technical explanation or analysi",
    "blueprint": json.dumps({
        "scene_goal": "Explain the mechanism or findings clearly and concisely.",
        "style_guide": "Maintain an objective and formal tone. Use precise terminolo",
        "structure": ["Definition", "Function/Operation", "Key Findings/Impact"],
        "instruction": "Organize the provided facts into the defined structure, adhe
    })
},
```

This is exactly the kind of predictable formatting you want in technical or scientific contexts. And finally, here's a casual summary blueprint, the kind of style you'd use for quick, easy-to-read overviews or explainer text:

```
{
    "id": "blueprint_casual_summary",
    "description": "A goal-oriented context for creating a casual, easy-to-read summar",
    "blueprint": json.dumps({
        "scene_goal": "Summarize information quickly and casually.",
        "style_guide": "Use informal language. Keep it brief and engaging. Imagine e",
        "instruction": "Summarize the provided facts using the casual style guide."
```



```
        })  
    }  
]
```

Here, there's no need for roles or rigid structure. Instead, the `style_guide` pushes the LLM toward brevity, informality, and an approachable tone—as if you were chatting with a colleague or explaining a concept to a friend.

To double-check that our blueprints are formatted properly, we can add a quick `print` function:

```
print(f"\nPrepared {len(context_blueprints)} context blueprints.")
```

The output confirms that we have defined three blueprints:

```
Prepared 3 context blueprints.
```

The takeaway here is that creating semantic blueprints is both powerful and demanding. Without them, you'd have to write a separate agent for every LLM task. That doesn't scale. Semantic blueprints allow you to capture procedural instructions once and reuse them endlessly. Instead of building drop-down menus for dozens of tasks, you can lean on a scalable, automated context engine.

With our blueprints defined, let's turn to the other side of the dual RAG setup: the knowledge base.

Data preparation: the knowledge base (factual RAG)

Our next task is to define the data for our knowledge base, the factual side of our RAG setup. This component holds relatively static information that the Researcher agent will search and synthesize into concise findings. And just to keep expectations realistic, you're designing this system, not the LLM. Behind the slick platforms, there's a lot of careful human work.

For this educational example, we'll use a small dataset about space exploration. It includes the space race and Apollo 11, the Juno mission, and Mars rovers. In this chapter, we'll represent the data as strings so we can focus on the architecture. Let's load the knowledge into a single raw string:

```
knowledge_data_raw = """  
Space exploration is the use of astronomy and space technology to explore outer space. The  
...  
Juno is a NASA space probe orbiting the planet Jupiter. It was launched on August 5, 2011,  
...  
A Mars rover is a remote-controlled motor vehicle designed to travel on the surface of Mar  
"""
```

We'll now build a small helper function to process this text into search-friendly pieces before embedding.

Helper functions for chunking and embedding

Language models don't process raw paragraphs—they process **tokens**, which are the model's way of splitting text into units it can understand. For example, the word *encoding* might be split into two tokens: *encod* and *ing*. The model uses these smaller parts to better understand word relationships.

We'll use the `cl100k_base` tokenizer, the standard recipe for OpenAI's newest models. This encoding has three advantages:

- It's clever about how it splits up words. Instead of seeing a word such as *encoding* as one single piece, it might break it down into smaller, meaningful parts such as *encod* and *ing*. This helps the model understand how different words relate to each other.
- It's built for speed and works very efficiently, which is perfect for chat-style models and for preparing text to be stored in a database.
- Using it ensures that the way you count and format your text matches exactly what the model expects to see and the way it was trained.

This consistency is critical. If our chunking doesn't match the embedding model's training, the semantic search results won't be accurate. It would be like trying to look up books in a library where half the cards use a different cataloging system.

Let's initialize the tokenizer:

```
# Initialize tokenizer for robust, token-aware chunking
tokenizer = tiktoken.get_encoding("cl100k_base")
```

RAG works best when information is split into focused chunks. Instead of cutting by character count, we'll cut by token count, which is more reliable for LLMs. Additionally, the program defines a chunk size of 400 tokens and an overlap of 50 tokens. The overlap helps maintain context between chunks. You will have to find the best chunk sizes and overlap sizes for each type of data you are processing.

The program now introduces a `chunk_text` function that ensures long documents are broken down into neat, overlapping windows of text that are easier for the model to handle and more accurate for semantic search:

```
def chunk_text(text, chunk_size=400, overlap=50):
    """Chunks text based on token count with overlap (Best practice for RAG)."""
    tokens = tokenizer.encode(text)
    chunks = []
    for i in range(0, len(tokens), chunk_size - overlap):
        chunk_tokens = tokens[i:i + chunk_size]
        chunk_text = tokenizer.decode(chunk_tokens)
        # Basic cleanup
        chunk_text = chunk_text.replace("\n", " ").strip()
        if chunk_text:
            chunks.append(chunk_text)
    return chunks
```

Once we have chunks, we need to convert them into embeddings. API calls can sometimes fail (for example, due to network latency), so we'll add a retry mechanism. The `@retry` decorator from the Tenacity library automatically retries failed calls using exponential backoff:

```
@retry(
    wait=wait_random_exponential(min=1, max=60),
    stop=stop_after_attempt(6)
)
```

We are not going to upload raw chunks. Therefore, we need to define the `get_embeddings_batch` function that sends a batch of texts to the OpenAI API. It returns the corresponding embeddings:

```
@retry(
    wait=wait_random_exponential(min=1, max=60),
    stop=stop_after_attempt(6)
)
def get_embeddings_batch(texts, model=EMBEDDING_MODEL):
    """Generates embeddings for a batch of texts using OpenAI, with retries."""
    # OpenAI expects the input texts to have newlines replaced by spaces
    texts = [t.replace("\n", " ") for t in texts]
    response = client.embeddings.create(input=texts, model=model)
    return [item.embedding for item in response.data]
```

Now that we can chunk text and reliably generate embeddings, we're ready to process our knowledge and context data and upload it to the vector store.

Process and upload (upsert) data

The final step in data preparation is processing our data and uploading it into Pinecone. This step is critical: until our embeddings live inside the vector store, the agents won't be able to retrieve them.

We'll handle this in two parts:

- **Context library:** Upload our semantic blueprints
- **Knowledge base:** Upload our factual data

Context library

Let's begin with the context library. This is where we're storing our semantic blueprints, the procedural instructions that guide *how* the system should write.

The program now iterates through the `context_blueprints` we defined earlier. For each blueprint, it embeds the description using `get_embeddings_batch`. The description is what we'll later search semantically, so it's what actually goes into the vector space. At the same time, the code builds a Pinecone vector object that contains the unique ID, the embedding values, and some metadata. The metadata includes the description again for reference and, crucially, the full `blueprint_json` instructions. These JSON instructions aren't embedded, but they're stored alongside the description so they can be retrieved when needed:

```
# --- 6.1. Context Library ---
print(f"\nProcessing and uploading Context Library to namespace: {NAMESPACE_CONTEXT}")
vectors_context = []
for item in tqdm(context_blueprints):
    # We embed the DESCRIPTION (the intent)
    embedding = get_embeddings_batch([item['description']])[0]
    vectors_context.append({
        "id": item['id'],
        "values": embedding,
        "metadata": {
            "description": item['description'],
            # The blueprint itself (JSON string) is stored as metadata
            "blueprint_json": item['blueprint']
        }
    })
```

Once the vectors are prepared, they are ready to be uploaded to Pinecone. This process is called **upserting**, which means *update if the ID already exists, otherwise insert as new*. We upload these vectors to the `NAMESPACE_CONTEXT` namespace using the `index.upsert` command:

```
# Upsert data
if vectors_context:
    index.upsert(vectors=vectors_context, namespace=NAMESPACE_CONTEXT)
    print(f"Successfully uploaded {len(vectors_context)} context vectors.")
```

If the code runs successfully, you'll see a confirmation telling you how many vectors were uploaded. At that point, the context library is live inside the `ContextLibrary` namespace and ready for the Librarian agent to query during runtime. Let's move on to the other half of the dual RAG setup: the knowledge base.

Knowledge base

Now, let's populate the knowledge base, which will hold our factual data. The first step is to run `chunk_text` on the raw knowledge string so it can be split into smaller, manageable pieces. These chunks are then processed in batches. For this example, we use a batch size of 100. The program iterates over each batch, generates embeddings using `get_embeddings_batch`, and prepares a vector structure for Pinecone. Each chunk is given a unique ID and paired with metadata that stores the original text. Here's how that looks in code:

```
# --- 6.2. Knowledge Base ---
print(f"\nProcessing and uploading Knowledge Base to namespace: {NAMESPACE_KNOWLEDGE}")
# Chunk the knowledge data
knowledge_chunks = chunk_text(knowledge_data_raw)
print(f"Created {len(knowledge_chunks)} knowledge chunks.")
vectors_knowledge = []
batch_size = 100 # Process in batches
for i in tqdm(range(0, len(knowledge_chunks), batch_size)):
    batch_texts = knowledge_chunks[i:i+batch_size]
    batch_embeddings = get_embeddings_batch(batch_texts)

    batch_vectors = []
    for j, embedding in enumerate(batch_embeddings):
        chunk_id = f"knowledge_chunk_{i+j}"
        batch_vectors.append({
            "id": chunk_id,
```

```

        "values": embedding,
        "metadata": {
            "text": batch_texts[j]
        }
    })
    # Upsert the batch
    index.upsert(vectors=batch_vectors, namespace=NAMESPACE_KNOWLEDGE)
    print(f"Successfully uploaded {len(knowledge_chunks)} knowledge vectors.")

```

When this runs, the output will track both the context library and knowledge base uploads. The `tqdm` progress bar shows each embedding step in real time. In this example, the context library confirmed that three blueprints were uploaded successfully. The knowledge base, being small, was split into just 2 chunks, which fit neatly into a single batch of 100. The upload then confirmed that both chunks were successfully stored:

```

Processing and uploading Context Library to namespace: ContextLibrary
[Tqdm output for 3/3 iterations]
Successfully uploaded 3 context vectors.
Processing and uploading Knowledge Base to namespace: KnowledgeStore
Created 2 knowledge chunks.
[Tqdm output for 1/1 iterations]
Successfully uploaded 2 knowledge vectors.

```

With both the context library and knowledge base populated, our vector store is now ready. The pieces are in place to build the context-aware system.

Building the context-aware system

Now that our data is prepared and stored in Pinecone, it's time to put everything together. In this section, we'll implement the context-aware system shown in *Figure 3.3*. The steps follow the `Context_Aware_MAS.ipynb` notebook, which builds the runtime execution phase of our architecture. We'll start by defining the specialist agents that handle core tasks, and then we'll look at how the Orchestrator coordinates them.

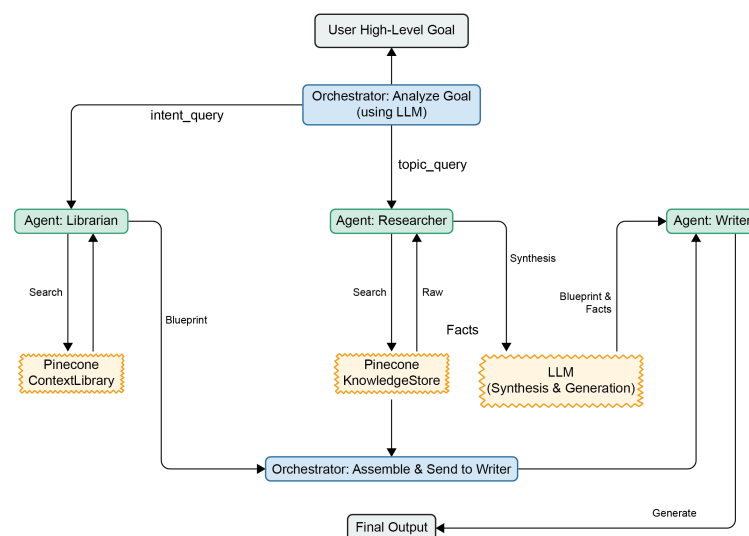


Figure 3.3: Context-aware MAS process

The flowchart shows how a high-level user goal is transformed into a structured workflow. It's a good reminder that generative AI doesn't "just work." Careful engineering and context management are what make the system reliable.

In the diagram, the Orchestrator (blue boxes) acts as the central coordinator, assigning work to the specialist agents (green boxes): the Librarian, the Researcher, and the Writer. These agents interact with external services (orange boxes) such as Pinecone (for knowledge and blueprint retrieval) and the LLM (for embedding, analysis, and generation).

With the big picture in place, let's turn to the details, starting with the agents themselves.

Defining the agents

The MAS relies on three specialized agents. Each agent performs a defined role:

- **Context Librarian:** Retrieves procedural instructions from the context library
- **Researcher:** Extracts and synthesizes factual data from the knowledge base
- **Writer:** Combines the procedural instructions with factual data to generate the final output

All agents communicate using the MCP. Let's start with the Librarian agent.

Context Librarian agent

The Context Librarian agent is responsible for retrieving the correct semantic blueprint from the vector store. This represents an advanced use of RAG, where procedural instructions are retrieved. Rather than pulling facts, we're pulling *how* to write:

```
def agent_context_librarian(mcp_message):
    """
    Retrieves the appropriate Semantic Blueprint from the Context Library.
    """
    print("\n[Librarian] Activated. Analyzing intent...")
    requested_intent = mcp_message['content']['intent_query']
    results = query_pinecone(requested_intent, NAMESPACE_CONTEXT, top_k=1)

    if results:
        match = results[0]
        print(f"[Librarian] Found blueprint '{match['id']}' (Score: {match['score']:.2f})")
        blueprint_json = match['metadata']['blueprint_json']
        content = {"blueprint": blueprint_json}
    else:
        print("[Librarian] No specific blueprint found. Returning default.")
        content = {"blueprint": json.dumps({"instruction": "Generate the content neutrally"})}

    return create_mcp_message("Librarian", content)
```

What's happening here is the Librarian semantically searches the `ContextLibrary` namespace using the user's intent (`intent_query`). When a matching blueprint is found, the JSON instructions are extracted and packaged in a new MCP message. This allows the Writer agent to follow dynamic procedural guidelines. We now need to build the Researcher agent.

As contained in the code, if the Librarian agent doesn't find a matching context blueprint, it returns a default set of instructions. The system is designed with a fallback mechanism to handle cases where a user's intent doesn't match any of the available semantic blueprints. Instead of failing, the Librarian agent creates and sends a default blueprint with a simple instruction: `Generate the content neutrally`. This ensures that the Writer agent still receives procedural guidance, allowing the system to produce a basic, unstyled response rather than stopping the process entirely.

Researcher agent

The Researcher agent focuses on extracting relevant factual information. The agent queries the knowledge base for the top relevant data chunks and synthesizes them into a concise summary that the Writer can use directly:

```
def agent_researcher(mcp_message):
    """
    Retrieves and synthesizes factual information from the Knowledge Base.
    """
    print("\n[Researcher] Activated. Investigating topic...")
    topic = mcp_message['content']['topic_query']

    results = query_pinecone(topic, NAMESPACE_KNOWLEDGE, top_k=3)

    if not results:
        print("[Researcher] No relevant information found.")
        return create_mcp_message("Researcher", {"facts": "No data found."})

    source_texts = [match['metadata']['text'] for match in results]
    system_prompt = """You are an expert research synthesis AI. Synthesize the provided source
    user_prompt = f"Topic: {topic}\n\nSources:\n" + "\n\n---\n\n".join(source_texts)

    findings = call_llm(system_prompt, user_prompt)

    return create_mcp_message("Researcher", {"facts": findings})
```

Here, the Researcher keeps the output tight and factual. Even in a larger system, this disciplined synthesis step prevents the Writer from drowning in raw text. We now need a Writer agent to write our content.

Writer agent

The Writer agent is where the two streams meet. It applies the semantic blueprint (procedural instructions) to the facts to generate the final, structured output:

```
def agent_writer(mcp_message):
    """
    Combines factual research with the semantic blueprint to generate final output.
    """
    print("\n[Writer] Activated. Applying blueprint to facts...")

    facts = mcp_message['content']['facts']
    blueprint_json_string = mcp_message['content']['blueprint']
    # The Writer's System Prompt incorporates the dynamically retrieved blueprint
    system_prompt = f"""You are an expert content generation AI.
    Your task is to generate content based on the provided RESEARCH FINDINGS.
    Crucially, you MUST structure, style, and constrain your output according to the rules def
```

```

--- SEMANTIC BLUEPRINT (JSON) ---
{blueprint_json_string}
--- END SEMANTIC BLUEPRINT ---

Adhere strictly to the blueprint's instructions, style guides, and goals. The blueprint de
"""

user_prompt = f"--- RESEARCH FINDINGS ---\n{facts}\n--- END RESEARCH FINDINGS ---\nGen
final_output = call_llm(system_prompt, user_prompt)

return create_mcp_message("Writer", {"output": final_output})

```

This agent dynamically programs the LLM using the blueprint as instructions, ensuring that the output respects both content and form. A small but important detail: the system prompt draws a hard line—**facts** determine *what* to say; the **blueprint** determines *how* to say it. That separation is what makes the system adaptable.

We will now build the Orchestrator that puts everything together.

Building the Orchestrator

The Orchestrator is the conductor of our system. Its job is to take a high-level goal, break it down into actionable queries, and coordinate the agents so they can produce the final output. Think of it as a project manager: it doesn't do the work itself but makes sure everyone else is in sync.

Goal analysis

The Orchestrator first decomposes the user goal into `intent_query` (the desired style, tone, or format) and `topic_query` (the factual subject matter). Here's how that looks in code:

```

def orchestrator(high_level_goal):
    """
    Manages the workflow of the Context-Aware MAS.
    """
    print(f"=== [Orchestrator] Starting New Task ===")
    print(f"Goal: {high_level_goal}")

    # Step 0: Analyze Goal (Determine Intent and Topic)
    # We use the LLM to separate the desired style (intent) from the subject matter (topic)
    print("\n[Orchestrator] Analyzing Goal...")
    analysis_system_prompt = """You are an expert goal analyst. Analyze the user's high
1. 'intent_query': A descriptive phrase summarizing the desired style, tone, or format
2. 'topic_query': A concise phrase summarizing the factual subject matter required (e.

Respond ONLY with a JSON object containing these two keys."""

    # We request JSON mode for reliable parsing
    analysis_result = call_llm(
        analysis_system_prompt, high_level_goal, json_mode=True
    )

    try:
        analysis = json.loads(analysis_result)
        intent_query = analysis['intent_query']
        topic_query = analysis['topic_query']
    except (json.JSONDecodeError, KeyError):
        return

```


Here, the Orchestrator calls the LLM itself to parse the high-level goal into structured JSON. If parsing fails, the function simply returns—this keeps the system from cascading into errors.

This `try...except` block effectively acts as a fallback mechanism: if the LLM produces invalid or incomplete JSON, the Orchestrator halts gracefully rather than passing corrupt data downstream. In production, you could further harden this step with structured prompting or schema validation, but for this prototype, the lightweight approach keeps the system resilient and easy to follow.

Agent coordination

Once the goal is broken down, the Orchestrator coordinates the flow of information between agents. It sends `intent_query` to the Librarian, `topic_query` to the Researcher, and then hands both results to the Writer:

The Orchestrator queries the Librarian for the context blueprint:

```
# Step 1: Get the Context Blueprint (Procedural RAG)
mcp_to_librarian = create_mcp_message(
    sender="Orchestrator",
    content={"intent_query": intent_query}
)
# display_mcp(mcp_to_librarian, "Orchestrator -> Librarian")
mcp_from_librarian = agent_context_librarian(mcp_to_librarian)
display_mcp(mcp_from_librarian, "Librarian -> Orchestrator")

context_blueprint = mcp_from_librarian['content'].get('blueprint')
if not context_blueprint: return
```

The Orchestrator then retrieves the factual knowledge from the Researcher:

```
# Step 2: Get the Factual Knowledge (Factual RAG)
mcp_to_researcher = create_mcp_message(
    sender="Orchestrator",
    content={"topic_query": topic_query}
)
# display_mcp(mcp_to_researcher, "Orchestrator -> Researcher")
mcp_from_researcher = agent_researcher(mcp_to_researcher)
display_mcp(mcp_from_researcher, "Researcher -> Orchestrator")

research_findings = mcp_from_researcher['content'].get('facts')
if not research_findings: return
```

Finally, the Orchestrator tasks the Writer with generating the output:

```
# Step 3: Generate the Final Output
# Combine the outputs for the Writer Agent
writer_task = {
    "blueprint": context_blueprint,
    "facts": research_findings
}

mcp_to_writer = create_mcp_message(
    sender="Orchestrator",
```

```

        content=writer_task
    )
    # display_mcp(mcp_to_writer, "Orchestrator -> Writer")
    mcp_from_writer = agent_writer(mcp_to_writer)
    display_mcp(mcp_from_writer, "Writer -> Orchestrator")

    final_result = mcp_from_writer['content'].get('output')

    print("\n=== [Orchestrator] Task Complete ===")
    return final_result

```

This pattern keeps responsibilities crystal clear: the Librarian retrieves procedural guidance, the Researcher gathers facts, and the Writer generates the final text by applying one to the other. Also, this architecture is solid because it's modular and built to scale. You get reliable, context-aware content because each agent has exactly one job and the system strictly follows the procedural instructions you give it. The Orchestrator acts like a good project manager—it directs the workflow but stays out of the weeds, letting the specialists do their jobs.

The real game-changer is separating the *what* from the *how*. We keep factual retrieval completely separate from the stylistic and procedural guides. This is a huge win because it means you can update your knowledge base without ever touching your style rules, and vice versa, which makes iterating incredibly fast.

And because the semantic blueprints are dynamic, you can swap out the system's style, tone, and structure for any given task. This flexibility is what makes the design so powerful for different applications. Ultimately, this isn't just an academic exercise; it's a practical blueprint for building smart, MASs that can actually handle complex, context-sensitive work.

Summary

We began this chapter by evolving the MAS from [Chapter 2](#). The core of our new architecture is the dual RAG approach. We created two separate databases within a single Pinecone index. The knowledge base stores factual information for the Researcher agent. The context library stores procedural semantic blueprints for the new Context Librarian agent. This separation allows the system to manage what it knows independently from how it acts.

We implemented the system in two parts. The first notebook prepared and uploaded the data into the two Pinecone namespaces. The second notebook defined the agentic workflow. We showed how an Orchestrator could analyze a user's goal. It delegates fact-finding to the Researcher and style retrieval to the Librarian. The Writer agent then uses both the facts and the stylistic blueprint to generate a constrained output. Our system can now dynamically control its generation process. We are ready to go to the next level and leverage what we built here to design a context engine.

Questions

1. Does the system just use one kind of RAG for everything? (Yes or no)
2. Is the context library just for storing facts, such as things about space exploration? (Yes or no)
3. Do the Researcher and the Librarian look for information in the same place? (Yes or no)

4. Does the Writer get to choose its own writing style? (Yes or no)
5. Is the Orchestrator's main job to do the actual writing? (Yes or no)
6. To find a style guide, does the system search for the entire JSON code? (Yes or no)
7. Is the system still using the fake data from the last chapter? (Yes or no)
8. Does the system use two completely separate databases for facts and style guides? (Yes or no)
9. Is the Librarian a new agent we just added? (Yes or no)
10. Does the Researcher give its info directly to the Writer? (Yes or no)

References

- Pinecone. (2024). *Retrieval-Augmented Generation (RAG)*. Pinecone. Retrieved from <https://www.pinecone.io/learn/retrieval-augmented-generation/>
- Pinecone. (2024). *Pinecone Documentation*. Pinecone. Retrieved from <https://docs.pinecone.io/>
- Gao, Y., Gao, Z., Zhang, Y., Li, S., Cai, D., Liu, Z., & Wang, S. (2023). Retrieval-Augmented Generation for Large Language Models: A Survey. *Retrieval-Augmented Generation for Large Language Models: A Survey*. arXiv preprint arXiv:2312.10997. Retrieved from <https://arxiv.org/abs/2312.10997>
- Sharma, C. (2024). *Retrieval-Augmented Generation: A Comprehensive Survey of Architectures, Enhancements, and Robustness Frontiers*. arXiv preprint arXiv:2406.00054. Retrieved from <https://arxiv.org/abs/2406.00054>
- Barnett, S., Baki, S., Crampton, J., et al. (2024). *A Survey of Retrieval-Augmented Generation (RAG) in the Era of Large Language Models*. arXiv preprint arXiv:2404.18023. Retrieved from <https://arxiv.org/abs/2404.18023>

Further reading

- Guu, K., Lee, K., Tung, Z., Pasupat, P., & Chang, M. (2020). REALM: Retrieval-Augmented Language Model Pre-Training. *REALM: Retrieval-Augmented Language Model Pre-Training*. arXiv preprint arXiv:2002.08909. Retrieved from <https://arxiv.org/abs/2002.08909>
- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. arXiv preprint arXiv:2005.11401. Retrieved from <https://arxiv.org/abs/2005.11401>

Get This Book's PDF Version and Exclusive Extras

Scan the QR code (or go to packtpub.com/unlock). Search for this book by name, confirm the edition, and then follow the steps on the page.



UNLOCK NOW

Note: Keep your invoice handy. Purchases made directly from Packt don't require one.

Table of contents collapsed